

Conclusion to REST

 [_ \(https://github.com/learn-co-curriculum/python-p4-conclusion-to-rest\)](https://github.com/learn-co-curriculum/python-p4-conclusion-to-rest)  [_ \(https://github.com/learn-co-curriculum/python-p4-conclusion-to-rest/issues/new\)](https://github.com/learn-co-curriculum/python-p4-conclusion-to-rest/issues/new)

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Conclusion

In this module, we covered the basics of REST. There is much more to keep in mind if you dive into a career in API development (covered a bit in this module's bonus lesson!), but you know more than enough to get started:

- REST is based on HTTP and makes use of its request methods (verbs) for routing.
- Flask supports REST natively and has an extension, Flask-RESTful, that forces routing based on HTTP request methods.
- If routing conventions are followed, REST APIs aren't so different from basic APIs! They make use of all of the same functionality under the hood.
- REST APIs can be used as standalone interfaces or backends for user-facing applications.
- Bonus: There are 6 architectural constraints to REST: uniform interface, client-server, cacheable, stateless, layered system, and, optionally, code-on-demand.

REST APIs are not always the solution to application-interfacing, but they should be used where they fit. Developers in all languages all around the world are familiar with REST and will be more likely to use your product if you make use of it in your design.

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means) ➞ (https://codewords.recurse.com/issues/five/what-restful-actually-means)
- [REST API Architectural Constraints](https://www.geeksforgeeks.org/rest-api-architectural-constraints/) ➞ (https://www.geeksforgeeks.org/rest-api-architectural-constraints/)
- [What is an API? - MuleSoft](https://www.mulesoft.com/resources/api/what-is-an-api) ➞ (https://www.mulesoft.com/resources/api/what-is-an-api)
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ➞ (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)